Module 16-1

Coding and Synthesizing an Example Verilog Design

cadence

This page does not contain notes.

Module Objective

In this module, you:

- Construct a sample Verilog RTL design for synthesis

Topics

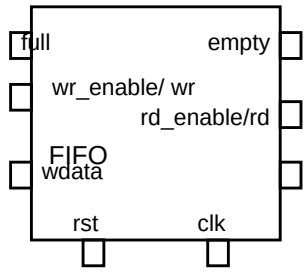
- FIFO specification
- FIFO implementation
  - Parameters
  - Ports
  - Variables
  - Functionality
  - Outputs
- Testbench design and implementation

cadence

This module applies the Verilog RTL for synthesis standards to construct a sample design.

FIFO Specification

- Parameterized for width and depth
  - Depth restricted to power of 2
- Asynchronous high-active reset
- Write/read synchronous to rising clock
- Ignores write when full
- Ignores read when empty



The sample design is a queue with a FIFO protocol, parameterized for width and depth, reset asynchronously and otherwise synchronized to a clock.

FIFO Implementation

Store FIFO contents in a register array.

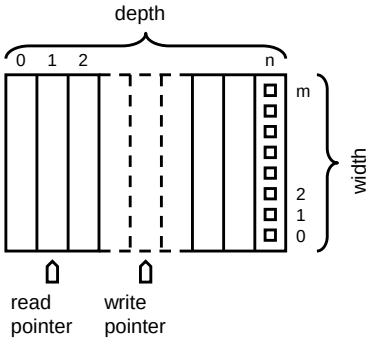
Access array with write/read pointers.

- For write:
  - Store data at current address
  - Increment pointer, maybe wrap
- For read:
  - Continuously drive current data
  - Increment pointer, maybe wrap

Derive the FIFO status from the pointer addresses.

Full – Addresses are equal and last operation was write.

Empty – Addresses are equal and last operation was read.



- Both pointers start at 0.
- Write pointer deposits value and increments.
- FIFO outputs data at read pointer.
- Read pointer increments to point at new data.
- Read pointer follows write pointer.
- Both pointers wrap around to array beginning.

Implement the FIFO by using a register array addressed by separate write and read pointers:

- Reset the write pointer to 0. Upon each write operation, if the FIFO is not already full, store the data at the current write address and increment the write pointer to point to the next location, wrapping back to the array beginning if necessary.
- Reset the read pointer to 0. Continuously present the array data at the current read address to the data output port. Upon each read operation, if the FIFO is not already empty, increment the read pointer to point to the next location, wrapping back to the array beginning if necessary.
- The read address thus follows the write address around the array. At any point where the read address is the same as the write address, you know that the FIFO is either empty or full. It is full if the most recent operation was to write and empty if the most recent operation was to read. Include a register to track whether the most recent operation was to write.

FIFO Module Structure

- Parameters
- Ports
- Variables
- Function

```
module fifo
#(
  // parameter declarations
 )
 (
  // port declarations
 );

  // variable declarations

  // functional code

endmodule
```

5

cadence

The basic building block of the design hierarchy is the Verilog module. Every module description starts with the module keyword followed by the module identifier (“fifo”) and ends with the endmodule keyword. The module definition name must be unique within the definitions name space.

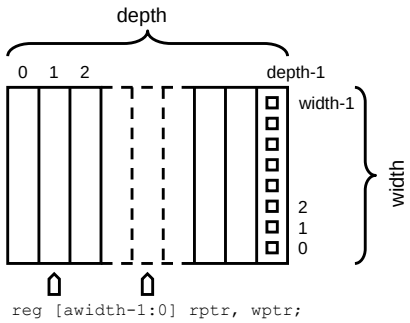
Following the module name is an optional module parameter port list, an optional list of ports or list of port declarations, and optional module items.

As for this design you will parameterized the input and output port width, you must declare the parameters before you declare the input and output ports of the module.

You must also declare all constants and variables before you use them.

FIFO Parameters

- Use the module **parameter** construct to define width and depth.
- Declare the parameters before you declare the ports.
- You can override module parameter values for each individual instance.

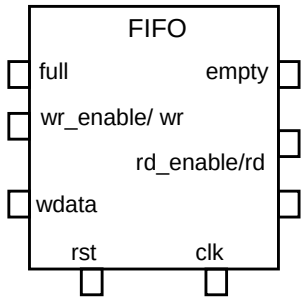



```
module fifo
#(
  // parameter declarations
  parameter awidth = 5,
  parameter dwidth = 8
 )
 (
  // port declarations
 );

  // variable declarations

  // functional code

endmodule
```

```
// 16 wide by 16 deep FIFO
fifo #(16,4) fifo16x16 (...
```


Use the module parameter construct to parameterize your module definition. You can override module parameter values on a per-instance basis. If you parameterize the input and output port widths, you must declare the parameters before you declare the input and output ports of the module. Module parameters are runtime constants that you can use wherever the syntax requires a literal value. You cannot change module parameters during the simulation.

FIFO Ports

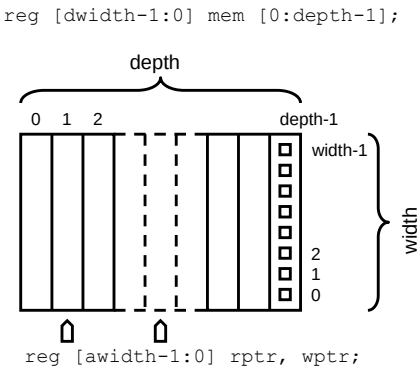



```
module fifo
#(
  // parameter declarations
  parameter AWIDTH = 5,
  parameter DWIDTH = 8
 )
 (
  // port declarations
  input clk, rst, wr_en, rd_en,
  input  [DWIDTH-1:0] data_in,
  output full, empty,
  output reg [DWIDTH-1:0] data_out
 );

  // variable declarations

  // functional code

endmodule
```


Utilize a module parameter to define the width of the data input and output ports. Parameterizing the data width allows you to modify it for each instance of the FIFO.

FIFO Variables

- Register array
- Write and read pointers
- Most recent operation type




```
module fifo
#(
  // parameter declarations
 )
 (
  // port declarations
 );

  // variable declarations
  localparam depth = 2**awidth;
  reg [dwidth-1:0] mem [0:depth-1];

  reg [awidth-1:0] wptr;
  reg [awidth-1:0] rptr;
  reg wrote;

  // functional code

endmodule
```

Verilog 2001
Power Operator

Utilize module parameters to define the width and depth of the register array and the width of the write address register and the read address register. Parameterizing the width and depth allows you to easily modify them for each instance of the FIFO.

FIFO Function

Asynchronous high-active reset

Write/read synchronous to rising clock

- For write:
  - Ignore write when full
  - Store data at current address
  - Increment pointer, maybe wrap
- For read:
  - Ignore read when empty
  - Increment pointer, maybe wrap

Track last operation

```
// functional code
always @(posedge clk or posedge rst)
begin
  if (rst)
    begin
      rptr <= 1'b0;
      wptr <= 1'b0;
      wrote <= 1'b0;
    end
  else
    begin
      if (rd && !empty)
         begin
          rdata <= mem[rptr];
          rptr <= rptr +1;
          wrote <= 0;
         end
      if (wr && !full)
         begin
          mem[wptr] <= wdata;
          wptr <= wptr +1;
          wrote <= 1;
         end
     end
end
```


Code a sequential procedure to describe the main functionality of the FIFO. The FIFO uses an asynchronous reset, so place the reset input as well as the clock input in the procedure event list. Remember that for synthesis all variables in an event list for a sequential procedure must be edge-qualified, so trigger the procedure by the positive edge of the reset and the positive edge of the clock.

While reset, initialize the write address and read address to location 0 and the tracking register to indicate that the most recent operation was to read.

Upon an active clock edge, check for a valid write or read operation:

- If the write enable is active and the FIFO is not full, store input data to the location of the current write pointer and increment the write pointer, wrapping back to the array beginning if necessary.
- If the read enable is active and the FIFO is not empty, increment the read pointer to address data in the next location, wrapping back to the array beginning if necessary.
- For this design, you can let the address registers wrap naturally. If the depth was not a power of 2 then you would need to explicitly wrap the address registers from their maximum value back to 0.

Upon an active clock edge, also update the tracking register.

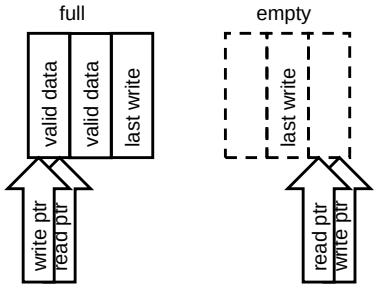
FIFO Outputs

Derive the FIFO status from the pointer addresses.

- Full – Addresses are equal and last operation was write
- Empty – Addresses are equal and last operation was read.

For read:

- Continuously drive current data.




```
// functional code
...


assign empty = (rptr == wptr) && !wrote;
assign full  = (rptr == wptr) &&  wrote;

endmodule
```

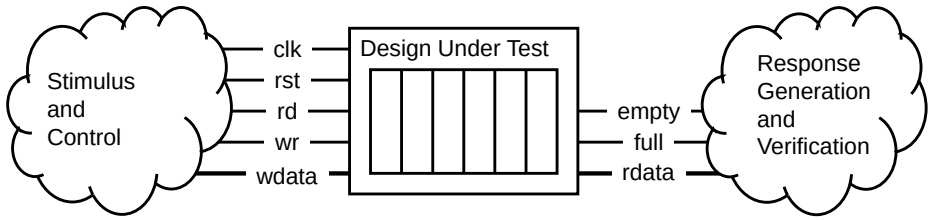

Continuously present to the data output port the register array data addressed by the read pointer.

Generate the FIFO status flags by comparing the read and write addresses. At any point where the read address is the same as the write address, you know that the FIFO is either empty or full. It is full if the most recent operation was to write and empty if the most recent operation was to read.

Note that signal “wrote” in the code is “last write” performed to the FIFO.

FIFO Testbench

- Instantiates and connects to the Design Under Test (DUT)
- Applies stimuli to the DUT input data and control ports
- Monitors the DUT output ports to verify correct behavior



To verify your FIFO model, you need to present it with stimulus and verify that its response is correct. In general, any testbench must do the following:

- Instantiate the Design Under Test (DUT) and connect its ports to local nets and variables.
- Apply stimuli to the input data and control (clock, reset) ports.
- Monitor the output ports to capture the response of the design to the applied stimuli.

A simple testbench records the stimulus and response vectors for subsequent analysis. Such analysis can for this design be a simple visual check of the transition data.

A more sophisticated testbench automatically compares the response to an internally generated response that is known to be correct, and reports whether the DUT failed or passed the test.

Modern testbenches tend to be extremely sophisticated. The majority of development effort is now utilized for verification.

Testbench Module Structure

- Constants
- Nets and variables
- Instances
- Function

```
module fifo_test;

  // Constant declarations

  // FIFO "signal" declarations

  // FIFO instantiation

  // FIFO stimulus

endmodule
```

Why no ports?

The top level of the simulation model is a testbench module. The top-level module does not have module parameters or ports because it is not instantiated (otherwise it would not be the top level). The top-level module declares design and test configuration constants, signals to connect to the DUT instance, and the DUT instance. A top-level module can declare procedures to provide stimulus and monitor response, and can instantiate additional test modules to also do some of that work

Testbench Declarations

- Testbench constants
- FIFO *signals*

```
module fifo_test;

  // Verilog-2001 local constants
  localparam dwidth = 8;
  localparam awidth = 5;
  localparam depth = 2 ** awidth;

  // Variables for FIFO inputs
  reg [dwidth-1:0] wdata;
  reg clk, wr, rd, rst;

  // Nets for FIFO outputs
  wire full;
  wire empty;
  wire [dwidth-1:0] rdata;

  // FIFO instantiation
  // FIFO stimulus

endmodule
```


Declare local constants for the FIFO data width and address width and clock period.

Using these constants, declare variables to provide stimulus and nets to receive the response.

Testbench DUT Instance

- The module definition name is fifo.
- Override the module parameters.
- The module instance name is dut.
- Use named port connections to bind FIFO ports to local *signals.*

```
module test_fifo;

  // Declarations

  // FIFO instantiation
  fifo
  #(
    .awidth(awidth),
    .dwidth(dwidth)
   )
  fifo
   (
    .wdata  (wdata),
    .rdata  (rdata),
    .clk      (clk),
    .rst      (rst),
    .wr     (wr),
    .rd     (rd),
    .full     (full),
    .empty    (empty)
   );

  // FIFO stimulus

endmodule
```

Verilog-2001 list of named parameter assignments

Verilog-1995 list of named port connections

cadence

Instantiate the fifo DUT and override its module parameters and connect its ports to local nets and variables.

Testbench Stimulus

```
module test_fifo;

  // Stimulus for reading

  initial
   begin
    $monitorb("%d",$time,,clk,,rst,,wr,,rd,,          // set up monitor for signal value change
                   wdata,,full,,empty,,rdata);
    clk = 0; wr = 0; rd = 0; rst = 0; wdata = -1;          // initialize
    @(negedge clk) rst = 1;                                // assert reset
    @(negedge clk) rst = 0;                                // deassert reset
    @(negedge clk) wr = 1;
    @(negedge clk) begin
     wr = 0;
     rd = 1;
    end
    @(negedge clk) begin
     rd = 0;
     wr = 1;
    end
   repeat (depth -1) @(negedge clk) wdata = wdata -1;
    @(negedge clk) begin
    wr = 0;
    rd = 1;
   end
   repeat (depth +1) @(negedge clk);
  $stop;
 end
always #10 clk = ~ clk;
endmodule
```

wr signal is made high to write wdata to FIFO.

rd is asserted so that FIFO that was previously written can be read.

wr signal is asserted so that WDATA can be written to the FIFO.

repeat loop is added to write the wdata content for (2 ** awidth-1) times, full signal can be seen high in the last clock pulse.

rd is assrted and kept high for (2 ** awidth+1) clocks for reading of FIFO, empty signal can be seen high in the last clock pulse.

cadence

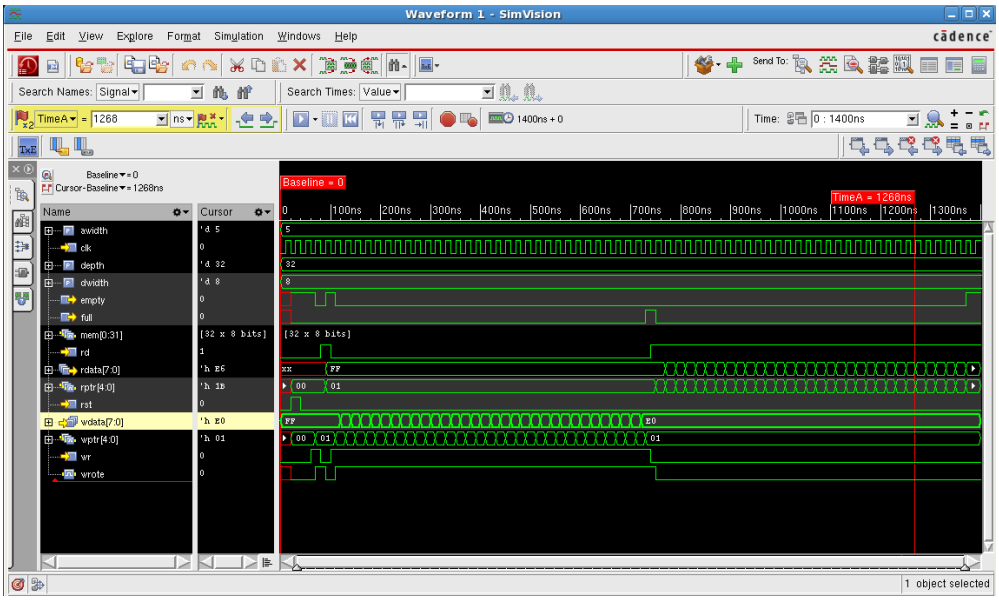
The $monitor family of system tasks output the value of their arguments at the end of each unique simulation time in which any of them has changed.

As the fifo DUT is synchronized to the rising clock edge, the stimulus is synchronized to the falling clock edge. Having done that, the stimulus can utilize blocking assignments without causing clock/data races.

As is typical for a testbench, the stimulus tests the “corner” conditions of the DUT algorithm.

The $stop system task stops the simulation.

Simulation Results



cadence

Here is a Waveform window of the Cadence SimVision graphical simulation analysis environment displaying the transition history of the testbench signals.

Module Review

1. What are module parameters?
2. What are local parameters?
3. How do you override the value of a module parameter?
4. Why do we size literals used in arithmetic operations?

cadence

This page does not contain notes.

Module Review Solutions

1. What are module parameters?
   - Module parameters are constants such as width and depth that you can override on a per-instance basis.
2. What are local parameters?
   - Local parameters are constants that you cannot override.
3. How do you override the value of a module parameter?
   - You override module parameter values as you instantiate the module by providing a list of parameter values.
4. Why do we size literals used in arithmetic operations?
   - Unsized literals are 32-bit values. A logic synthesis tool may initially assume that you need for example a 33-bit adder when adding an unsized literal to a much smaller value.

cadence

This page does not contain notes.

Module 16-2

Using System Tasks and System Functions

cadence

This module examines system tasks and system functions commonly used for stimulus generation and debugging. Later modules will revisit some of these in more detail and introduce some more.

Module Objective

In this module, you:

- Use system tasks and functions for stimulus generation and debug

Topics

- Displaying messages
- Getting simulation time
- Formatting the time display
- File input and output
- Applying stimulus from a file
- Controlling the simulation
- Passing real values through ports
- Getting command-line values
- Dumping value-change data
- Checkpointing the simulation

cadence

This page does not contain notes.

Displaying Messages: `$display` and `$write`

The `$display` and `$write` system tasks print values to the standard output.

- `$display` appends a newline
- `$write` does not

The default base is decimal. Verilog supports other bases:

- `$displayb`
- `$displayo`
- `$displayh`

Arguments can include formatting strings, which can include escaped characters:

`\n  \t  \ddd  \\  \"`

```
module disp_wr;
 reg [7:0] var1, var2, var3;
 initial
  begin
   var1 = 8'h0F;        //  15
   var2 = 8'b01010101; //  85
   var3 = 8'b11001100; // 204
   $display (var1,var2,var3);
   $displayb (var1,, var2,, var3);
   $displayh (var1);
   $write (var1);
   $writeh (" var2(hex) is ", var2);
   $writeo ("\n var3(oct) is ", var3);
   $write  ("\n");
  end
endmodule
```

Null Argument

String Argument

Output sized to variable – 8 bits always written in decimal as 3 characters

```
 15 85204
00001111 01010101 11001100
0f
15 var2(hex) is 55
 var3(oct) is 314
```

Consecutive commas replaced by single space

cadence

The $display and $write system tasks are identical, except that $display adds a newline character to the end of every output, and $write does not.

The $display and $write system tasks support multiple default bases. The default base for $display and $write is decimal. Append the b character to make the default base binary, o for octal and h for hexadecimal. You can embed formatters in string arguments to override that default radix for individual subsequent arguments. Later pages demonstrate this.

The display width depends on the base and the maximum size of the variable. For example, the maximum value of an 8-bit vector is 256, which requires 3 decimal characters, so every 8-bit value written in decimal requires 3 characters. To display an 8-bit vector requires 8 characters in binary, 3 in octal and 2 in hexadecimal. The decimal radix replaces leading zeros by spaces, while the other radices display the leading zeros. When using a format string, you can override this automatic sizing. Later pages demonstrate this.

You can include string arguments to separate values and to make the output more meaningful. You can also separate values by including null arguments. A null argument is an extra comma between arguments. For a null argument, the simulator displays a single space.

Formatting Text Output

Each format specifier formats one argument value.

- Usually grouped into one first argument format string.
- Can be distributed among the task arguments.
- Matched to argument values in order – one to one match.
  - More values than format specifiers simply use default format.
  - More format specifiers than values is an error.

```
module disp_wr_fmt;
 reg [7:0] var1, var2, var3, var4;
 initial
  begin
   var1 = 8'h0F;        //  15
   var2 = 8'b01010101; //  85
   var3 = 8'b11001100; // 204
   $display ("%b", var1);
   $display ("var2 binary: %b \t", var2, "hex: %h", var2);
   $display ("%h \t %h \t %h", var1, var2, var3);
   $write ("var3 is hex %h \t octal %o \n", var3, var3);
  end
endmodule
```

```
00001111
var2 binary: 01010101     hex: 55
0f        55        cc
var3 is hex cc     octal 314
```


cadence

You can embed formatters in your string arguments to, among other things, override the default base. A formatter can apply to only one subsequent argument, but other than that, you can group the formatters any way you want. Many people put them all in one first argument string, and some people insert individual format strings just prior to the argument they apply to.

When using format strings, you must be very careful to not apply the formatters to null arguments. You might want to adopt the convention that you do not use both in the same statement.

Within any character string literal, you can place escaped character sequences to represent special characters such as newline and tab characters.

The 1st display statement overrides the default base to display the value of the var1 argument in binary. Format characters can be in either case, but most people use the lower case as is done here. The var1 argument is an 8-bit vector, so displays in 8 characters.

The 2nd display statement displays the value of var2 in binary and hexadecimal, with a space and tab between them.

The 3rd display statement displays the value of var1, var2 and var3 in hexadecimal with a space and tab and an extra space between them.

The 4th display statement displays the value of var3 in hexadecimal and octal, with a space and tab and extra space between them.

Reference: Format Specifications

Format	Description	Example
%b %B	Display in binary format	$display("%b",1'b1); // 1
%o %O	Display in octal format	$display("%o",3'o7); // 7
%d %D	Display in decimal format	$display("%0d",4'd9); // 9
%h %H	Display in hexadecimal format	$display("%h",4'hF); // f
%c %C	Display in ASCII character format	$display("%c",65); // A
%s %S	Display as a string	$display("%s","foo"); // foo
%e %E	Display real in exponential format	$display("%e",3.1); // 3.100000e+00
%f %F	Display real in float format	$display("%f",3.1); // 3.100000
%g %G	Display real using %e or %f	$display("%g",3.1);
%l %L	Display library binding information	$display("%l"); // work.test
%m %M	Display hierarchical name	$display("%m"); // top.test1
%t %T	Display in current time format	$display("%t",$time); // 1.1 ns
%v %V	Display net signal strength	$display("%v",n1); // St1
%u %U	Unformatted 2 value binary data	
%z %Z	Unformatted 4 value binary data	

The integer formatters by default display values in a width sufficient to contain the maximum value the expression size can represent. For the decimal format they display leading spaces and for the other formats they display leading zeros. This tabular format makes vertically long displays more readable, but often unnecessarily wastes horizontal display space. An integer, for example, always requires 10 spaces no matter how small the value. You can override this automatic sizing. Placing a 0 character between the % character and the radix character reduces the displayed width to only the width actually required to display the value. Unlike the C language, in Verilog, you cannot specify the display format more precisely.

The binary formatter always displays high-impedance or unknown bits with lower case z and x characters. The octal and hexadecimal formatters use upper case Z and X characters where the bits that the character represents contain at least one bit with a binary value and at least one bit with a high-impedance or unknown value. In this case, an unknown bit trumps any high-impedance bits to display an upper case X. The decimal formatter does not try to compute the bit range in this case and displays the entire number using uppercase Z or uppercase X.

The %l formatter displays the library binding information – the module definition name preceded by a period (.) and the name of the library in which the elaborator found the compiled definition.

The %m formatter displays the hierarchical scope of the instance containing the display system task.

The %t formatter displays the argument according to the format you set with the $timeformat system task. With this task, you set the format width and precision for a time argument. You can use this formatter for any integer value if you want to thoroughly confuse your co-workers.

The %v formatter displays the strength and value of a net. Net drivers have associated with them one of eight strengths, that the simulator uses to resolve the strength and value of the net.

The %u and %z formatters write binary data, presumably to a file, for an application that reads binary data.

Reference: Escape Sequences in Format Strings

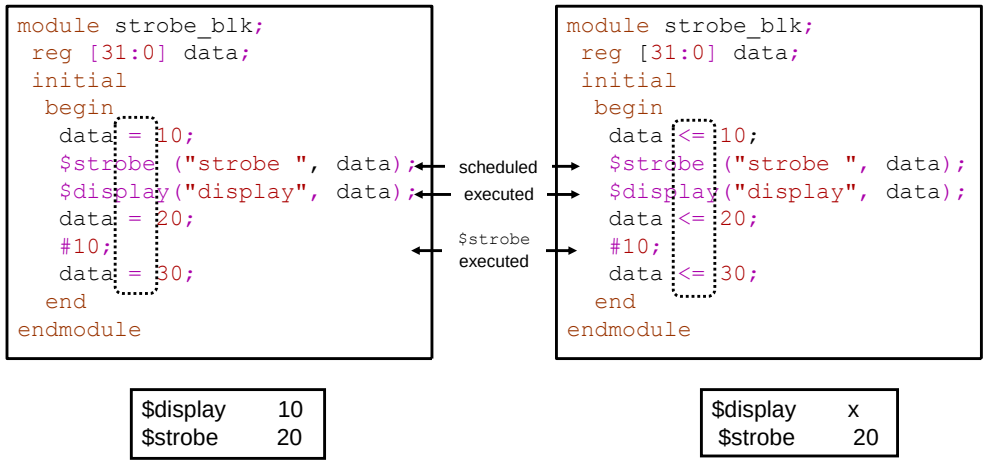
Argument	Description
`\n`	The newline character
`\t`	The tab character
`\\`	The `\` character
`\"`	The `"` character
`\ddd`	A character specified by 1 to 3 octal digits
`%%` [2001]	The `%` character

Here is the full set of escape sequences provided by the IEEE Std 1364-2001 Verilog HDL. The Verilog-2001 update added the escaped percent (%) character. Note that it is escaped by another percent character and not by a backslash (\) character.

Displaying Messages: $strobe

The $strobe system task is similar to the $display system task

- Execution of $strobe is scheduled for the end of the current time “slice”.



The $strobe system task evaluates and displays its arguments at the end of the current simulation time, and is otherwise identical to the $display system task. Like the $display system task, it is available in four versions with four different default bases.

The left example uses blocking assignments, so the $display system task displays the assigned value 10 and the $strobe system task displays the later assigned value 20.

The right example uses nonblocking assignments, so the $display system task displays the not-yet-assigned default unknown value (x) and the $strobe system task displays the later assigned value 20.

Getting Simulation Time: `$stime`, `$time`, `$realtime`

The `$time`, `$stime` and `$realtime` system functions return the simulation time:

- `$time`
  - Returns time as a 64-bit integer
- `$stime`
  - Returns time as a 32-bit integer
- `$realtime`
  - Returns time as a real number

The simulator scales returned time values to the timescale of the calling module.

```
`timescale 1 ns / 100 ps

module time_sysf;
 reg [5:0] data;
 initial
  begin
   #10;
   data = 6'd20;
   $display ($time,, data);
   #10.4;
   data = 6'd30;
   $write ("time: %0d", $time);
   $write (" realtime: %f", $realtime);
   $write (" data: ", data, "\n");
  end
endmodule
```

%0d removes leading spaces

```
                  10 20
time: 20 realtime: 20.4 data: 30
```


cadence

The `timescale directive specifies the timescale units for following modules.

The $time system function returns 64-bit simulation time scaled to the time scale of the calling module and then if needed rounded to an integer value.

The $stime system function returns the lower 32 bits of the $time value. This might not for obvious reasons represent the actual simulation time. For short simulations, you can use $stime to display a value that takes ten spaces to display instead of twenty.

The $realtime system function returns 64-bit simulation time scaled to the time scale of the calling module and converted to a real value.

The $display statement displays 64-bit simulation time and a 6-bit data value separated with a space character.

The $write statements display formatted 64-bit simulation time followed by time as a real number followed by the data value, each separated by a space character.

Displaying Messages: $monitor

The $monitor system task continuously monitors nets and variables:

- It acts like $strobe when any argument (other than $time) changes value.
- It supports the same default bases and formatters as the $strobe system task.
- Only one $monitor task can be active at a time.
  - A new $monitor replaces any current $monitor.
- You can disable monitoring with **$monitoroff** and re-enable it with **$monitoron**.

```
module time_monitor;
 reg [7:0] var1, var2, var3, var4;
 initial
  begin
   $monitor("%0d \t %h %h %h",
    $time, var1, var2, var3);
   var1 = 8'h0F;
   var2 = 8'b0101_0101;
   var3 = 8'b1100_1100;
   #5;
   var1 = 8'h80;
   #8;
   var2 = 8'h66;
   var3 = 8'h77;
   #4;
   var1 = 8'h55;
  end
endmodule
```

```
0          0f 55 cc
5          80 55 cc
13         80 66 77
17         55 66 77
```


cadence

The $monitor system task, once invoked, continually monitors its arguments, and displays their values at the end of the time “slice” in which any argument changed.

Thus the $monitor system task, like the $strobe system task, involves the scheduler.

Like the $strobe system task, it is available in four versions with four different default bases.

At most one monitor is active at a time. A new call to $monitor replaces the list of monitored signals. The simulator ceases monitoring the previous list of signals and instead monitors the new list of signals.

You can control monitoring with the $monitoroff and $monitoron system tasks. You can use these system tasks to monitor signal values only between certain time intervals of the simulation.

Formatting the Time Display: `$timeformat`

The `$timeformat` system task sets the display format for the `%t` formatter.

- Controls units, precision, suffix and field width.
- Format is consistent for `$time`, `$stime` and `$realtime`.
- Format is consistent for all timescales.
  - `` `timescale `` is required.
- Supported by all variants of `$display`, `$monitor`, `$strobe` and `$write` tasks.

```
`timescale 1 ns / 10 ps
module time_fmt;
 wire o1;
 reg in1;
 assign #9.53 o1 = ~in1;
 initial
  begin
$display("time \t realtime \t in1 \t o1");
   $timeformat(-9, 2, " ns", 10);
   $monitor("%0d \t %t \t %b \t %b",
    $time, $realtime, in1, o1);
   in1 = 0;
   #10;
   in1 = 1;
   #10;
  end
endmodule
```

```
time      realtime         in1      o1
0            0.00 ns       0        x
10           9.53 ns       0        1
10          10.00 ns       1        1
20          19.53 ns       1        0
```


cadence

You use the $timeformat system task to specify how the %t formatter displays time values.

The $timeformat system task takes four arguments:

- The 1st argument is an integer giving the time unit to which to scale the displayed time. The argument value must be between 0, meaning seconds, and –15, meaning femtoseconds.
- The 2nd argument is an integer giving the precision for the display, that is, how many digits to display after the decimal point.
- The 3rd argument is a string to append to the display. People typically use this string to show the time units they scaled the time to.
- The 4th argument is an integer giving the minimum field width for the display. The display inserts leading spaces to force this minimum field width.

The $timeformat system task in this example scales simulation time to nanoseconds, displays it with two fractional digits, follows it with a string indicating its time units, and displays it with a ten-character minimum field width.

The IEEE Std 1364-2001 Verilog HDL Section 17.3.2 states that the $timeformat applies to “all %t formats specified in all modules that follow in the source description until another $timeformat system task is invoked”.

File Output: Opening Files with `$fopen`

You can write to files:

- Open the file with `$fopen`.
  - Returns a 32-bit unsigned integer multi-channel descriptor (MCD) with one bit set to 1.
    - Returns 0 if unsuccessful.
  - Each successful `$fopen` returns a different MCD bit set to 1.
  - Channel 0 is pre-opened to the standard output.
  - You can open up to 30 additional channels.
    - Bit 31 is reserved
      always 0
- Close the channel with `$fclose`.
  - Allows channel reuse.

```
module file_write;
 integer data_chan, warn_chan;
 reg [7:0] var1, var2;
 initial
  begin
   data_chan = $fopen("data.txt");
   if (!data_chan) $finish;
   $displayb ("data_chan ", data_chan);
   warn_chan = $fopen("warn.txt");
   if (!warn_chan) $finish;
   $displayb ("warn_chan ", warn_chan);
   $fmonitor (data_chan, var1, var2);
   // do stuff ...
   $monitoroff;
   $fclose(data_chan);
  end
endmodule
```

MCD Values

```
<stdout>  00000000000000000000000000000001
data_chan 00000000000000000000000000000010
warn_chan 00000000000000000000000000000100
```

29

cadence

Use the $fopen system function to open a file. The $fopen system function returns a 32-bit unsigned multichannel descriptor (MCD) uniquely associated with the file. The system function returns 0 if it could not open the file for writing.

You must save the returned MCD as an integer or 32-bit reg vector.

Think of the MCD as a set of 32 flags, where each flag represents a single output channel:

- The most significant bit of an MCD is reserved for purposes we will discuss later and is always 0.
- The least significant bit of an MCD is reserved for the standard output. Most simulators also write this output to a log file.
- You can open up to 30 additional channels and simultaneously write to any combination of the open channels.

Use the $fclose system task to close the channels specified in its MCD argument. The $fclose system task closes the specified channels. Subsequent calls to the $fopen system function reuse those released channels.

File Output: Writing to Files

The $fdisplay, $fmonitor, $fstrobe and $fwrite set of system tasks take an MCD first argument and can simultaneously write to multiple channels.

```
reg [7:0] x, y;
integer datafile, timefile, bothfile;
initial
 begin
  datafile = $fopen("data.txt");
  timefile = $fopen("time.txt");
  bothfile = datafile | timefile;
  x = 77;
  y = 2;
  #50;
  $fdisplay(datafile, "X-axis %d\nY-axis %d", x, y );
  $fdisplay(timefile, "%0d X = %0d Y = %0d", $time, x, y );
  $fdisplay(bothfile, "Output to both files");
 end
```

MCD Values

```
datafile 00000000000000000000000000000010
timefile 00000000000000000000000000000100
bothfile 00000000000000000000000000000110
```

data.txt

```
X-axis 77
Y-axis  2
Output to both files
```

time.txt

```
50 X = 77 Y = 2
Output to both files
```


The four formatted display tasks ($display, $write, $monitor, and $strobe) have counterparts that write to a specific set of channels rather than to just the standard output.

The counterparts have an f prefix character and take an extra MCD first argument. The MCD can be any arbitrary expression that results in a 32-bit unsigned integer value. This value determines which open files the system task writes to. You simultaneously write multiple channels by bitwise ORing existing MCDs. You can set bit 0 to also write to the standard output.

This example opens the data.txt file and the time.txt file. It then bitwise ORs the two MCDs to create a third MCD representing both channels. It later writes to the channels individually and simultaneously.

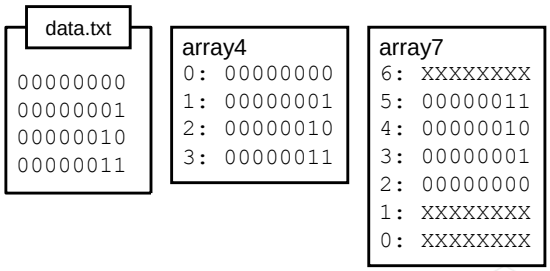
File Input: `$readmemb` and `$readmemh`

Verilog system tasks read from a file into a 1-D array of reg vector:

- Binary `$readmemb`
- Hexadecimal `$readmemh`

You can optionally specify start and end addresses (data at these addresses must exist in the file).

```
module readfile;
 reg [7:0] array4 [0:3];
 reg [7:0] array7 [6:0];
 initial
  begin
   $readmemb("data.txt", array4);
   $readmemb("data.txt", array7, 2, 5);
  end
 // other procedures ...
endmodule
```



The $readmemb and $readmemh system tasks load data from a text file into an array. Depending on the task, you provide the data in a binary radix or a hexadecimal radix. The system task has no versions to accept octal data or decimal data.

- The 1st argument is the data file name.
- The 2nd argument is the array to receive the data.
- The 3rd argument is an optional start address, and if you provide it, you can also provide –
- The 4th argument optional end address.

The data file itself can optionally specify to what addresses its data belongs.

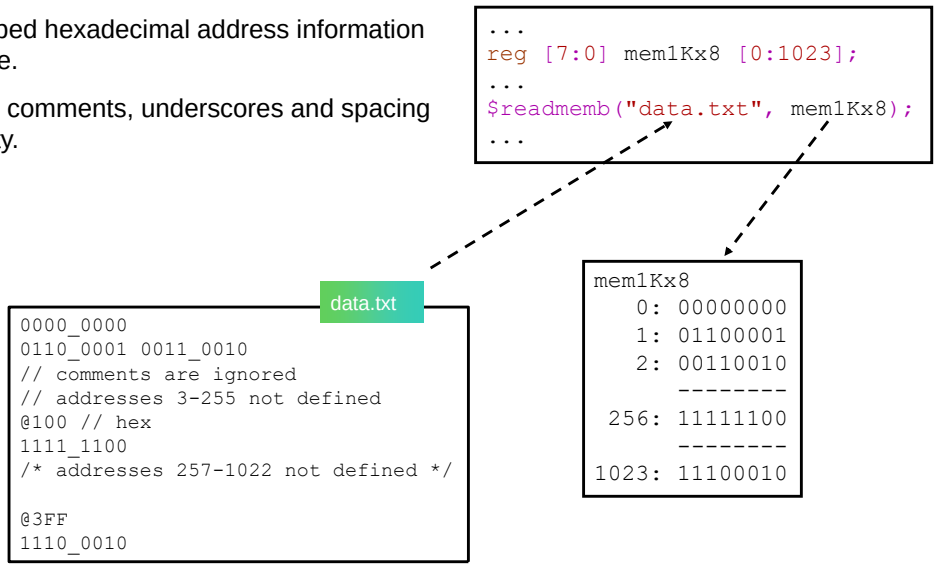
If the memory addresses are not specified anywhere, then the system tasks load file data sequentially from the left memory address bound to the right memory address bound. The year 2005 standard changes this default loading order to be from the lowest address toward the highest address.

If the file does not specify addresses for its data and the system task specifies a starting address, then the system tasks load file data sequentially from that starting address toward the specified ending address, or if no ending address is specified, then toward the right memory address bound for Verilog 2001 tools and toward the high memory address bound for Verilog 2005 tools.

If the file does specify addresses for its data, then if the system task specifies an address range, that address range must encompass and exhaust the available data.

File Input Data Format

- You can embed hexadecimal address information in the text file.
- You can use comments, underscores and spacing for readability.



Here are the things that a data file can contain:

- White space, including spaces, newlines, tabs, and formfeeds.
- Comments, which can be single line or multiline.
- Binary numbers or case-insensitive hexadecimal numbers, which:
  - If address values, are hexadecimal digits following an at (@) character.
  - If data, are a sequence of binary or hexadecimal digits without base or size information but that might embed underscore (_) characters for readability.

This example data file contains 8-bit binary data for only the addresses 0, 1, 2, 256 and 1023. A system task call that specifies an address range must include these addresses in that range.

Enhanced C-Style File I/O

Verilog-2001 adds C-like file system operations to the Verilog language.

- With added type option, **`$fopen`** returns a 32-bit File Descriptor (FD):
  - Type option is access, e.g., `r`, `w`, `a`
  - FD has bit 31 set to differentiate it from an MCD
  - Can represent 2**31 channels but never multiple channels
  - Channels 0, 1, 2 already opened to `stdin`, `stdout`, `stderr`
- Counterparts exist to almost all C file system functions:
  - Writing characters, lines, binary, or formatted
    - **`$fdisplay $fmonitor $fstrobe $fwrite $fflush`**
  - Reading characters, lines, binary, or formatted
    - **`$fgetc $ungetc $fgets $fread $fscanf`**
  - Manipulating the file pointer
    - **`$fseek $ftell $rewind`**

The Verilog 2001 update added support for C-style file I/O routines, thus providing the capability to input something more than just a memory data. These routines utilize a 32-bit unsigned integer file descriptor. A file descriptor represents a single channel. The file descriptor has the most significant bit set to 1 to differentiate it from a multichannel descriptor, which has the most significant bit set to 0. As in C, the first three channels are pre-opened to the standard output, input, and error files.

The enhanced C-style file I/O re-uses the existing output system tasks and provides new system tasks for input and for file pointer manipulation.

Reference: Enhanced C-Style File I/O

System Task/Function	Returns	Description
`$fopen( "filename", type ) ;`	fd	Opens a file
`$fclose( fd ) ;`		Closes a file
`$ferror( fd, str ) ;`	errcode	Gets error code and description
`$fgetc( fd ) ;`	char	Gets a character
`$ungetc( c, fd ) ;`	errcode	Ungets (puts back) a character
`$fgets( str, fd ) ;`	errcode	Gets a string
`$fflush( [ fd ] ) ;`		Flushes output buffer(s)
`$fread( reg, fd ) ;`	errcode	Reads binary data to a reg
`$fread( mem, fd [ , [start] [ , [count] ] ] );`	errcode	Reads binary data to a reg array
`$fscanf( fd, format, args ) ;`	errcode	Reads formatted data from a file
`$ftell( fd ) ;`	position	Gets the file pointer position
`$fseek( fd, offset, operation ) ;`	errcode	Repositions the file pointer
`$rewind( fd ) ;`	errcode	Rewinds the file pointer
`$sscanf( str, format, args ) ;g`	errcode	Reads formatted data from a string
`$sformat( reg, format, args ) ;`	length	Formats data to a string
`$swrite( reg, args ) ;`		Formats data to a string

cadence

This page intends to make you aware of what is available but encourages you to refer to the Verilog standard for full and complete documentation. The system tasks are similar, but not identical, to the C standard I/O library routines.

Example for Reading from a File

```
module testbench;
reg [8:1] result;
integer data_chan;

initial
 begin
 data_chan = $fopen("source.dat", "rb");

 while (result !== 255)
  begin : get_char
   result = $fgetc(data_chan);
   if (result == 255)
    begin
     $display ("Finished!");
     //Prevent $display printing out EOF
     disable get_char;
    end
   $display("%s %0d %b", result, result, result);
  end

 $fclose(data_chan);
 end
endmodule
```

source.dat

```
_!@%^&
abcZ
```

```
_  95 01011111
!  33 00100001
@  64 01000000
%  37 00100101
^  94 01011110
&  38 00100110

  10 00001010
a  97 01100001
b  98 01100010
c  99 01100011
Z  90 01011010

  10 00001010
Finished!
```

Type field determines access mode.

Result is set to EOF (-1) if an error occurs.

- `r` or `rb` – read (from existing file)
- `w` or `wb` – write (to new file)
- `a` or `ab` – append (to existing file)
- `r+`, `r+b`, `rb+` – read/write (existing)
- `w+`, `w+b`, `wb+` – read/write (new)
- `a+`, `a+b`, `ab+` – append (new or old)

cadence

An example for reading from a file using some of the C-style File I/O.

Simulation Control: `$finish` and `$stop`

- The `$stop` system task interrupts simulation and enters the interactive mode.
  - You can continue the simulation from the stop point.
- The `$finish` system task terminates the simulation and exits the simulator.

```
task expect (input [7:0] resp, exp);
 if (resp !== exp)
  begin
   $display("want %b - got %b",
            exp, resp);
   $display("TEST FAILED");
   $stop;
  end
endtask

initial
 begin
  wait (empty);
  $display("Data buffer empty");
  $display("TEST COMPLETE");
  $finish;
 end
```


The $stop system task interrupts the simulator and lets you enter commands interactively. You can continue the simulation from the stop point.

The $finish system task terminates the simulation.

This example defines a task to compare the actual response with the expected response and interrupt the simulation if they are not identical. If the test completes successfully then the initial block detects the completion and terminates the simulation.

Module Summary

You should now be able to use system tasks and functions for stimulus generation and debug.

This module discussed:

- Displaying messages with **$display**, **$write**, **$strobe**, and **$monitor**
- Getting simulation time with **$time**, **$stime**, and **$realtime**
- Formatting the time display with **$timeformat**
- File input and output with **$fopen**, **$fclose**, etc.
- Applying stimulus from a file with **$readmemb** and **$readmemh**
- Controlling the simulation with **$finish** and **$stop**
- Checkpointing the simulation with **$save** and **$restart**

cadence

You can now use system tasks and system functions for stimulus generation and debug.

This module discussed a variety of system calls your testbench can make to support stimulus generation and debug.

Module Review

1. What output system task would you use to capture the “steady-state” settled values of a collection of nets and variables whenever any one or more of them transition?
2. Explain briefly the difference between **$stop** and **$finish**.

cadence

This page does not contain notes.

Module Review Solutions

1. What output system task would you use to capture the “steady-state” settled values of a collection of nets and variables whenever any one or more of them transition?
   - This is probably most easily done with the **$monitor** system task.
2. Explain briefly the difference between **$stop** and **$finish**.
   - The **$stop** system task interrupts simulation, from whence you or a script can continue it. The **$finish** system task irrevocably terminates simulation.

This page does not contain notes.